Projet Semantic Web and Ontologies - Pierres précieuses

Owen BONSERGENT - Yeicy PAZ CORDOBA

October 2026

Contents

1 Introduction et notice IA **2**

2 Étapes **3**
2.1 Explication du choix du domaine 3
2.2 Modélisation : classes et propriétés 3
2.3 Démonstration du raisonnement OWL 5
2.4 Décidabilité `SWRL` 7
2.5 Validation `SHACL` 10
2.6 Bonnes pratiques en Web Sémantique 11

3 Conclusion **15**

1 Introduction et notice IA

Ce document fait office de rapport pour le projet effectué dans le cadre du cours Semantic Web and Ontologies du Master Intelligent Systems and Applications (ISA). Dans ce projet a été designé une ontologie sur le sujet des pierres précieuses, gemmes et minéraux rares. Cette ontologie présente : du raisonnement OWL, du raisonnement basé sur des règles SWRL et de la validation de contraintes SHACL.
Conformément au sujet, l'utilisation de l'IA a été limitée à moins de 50% de la production totale. L'IA a été utilisée pour les sujets suivants : débug, rappels / aide à la compréhension de sujets du cours, vérification du respect de l'ontologie vis-à-vis du sujet et génération de tableaux d'avancement sur le projet.

2 Étapes

2.1 Explication du choix du domaine

Le domaine "pierres précieuses, gemmes et minéraux rares" possède de forts liens entre les différents éléments le composant. Relié à la minéralogie globale et la composition chimique, il permet de mettre en avant une grande quantité de règles et relations intéressantes pour une ontologie. Ce projet met ainsi en avant la majorité des règles vues dans le cadre de ce cours.

2.2 Modélisation : classes et propriétés

L'ontologie (qui utilise dans sa version finale le préfixe *gemmes:*) modélise les pierres naturelles, leur composition chimique et leur origine géographique. Elle contient environ 60 classes et 66 individus, ajoutées au fil de sa création pour s'adapter aux contraintes du sujet. Les primitives utilisées se répartissent entre RDF, RDFS et OWL.

RDF et RDFS

Toutes les entités sont déclarées par des triplets `rdf:type` (`owl:Class`, `owl:ObjectProperty`, `owl:NamedIndividual`, etc.).
RDFS structure le schéma :

- `rdfs:subClassOf` définit la hiérarchie : `PierreNaturelle` se spécialise en `Mineral`, `Mineraloide`, `Roche`, `Gemme` et `Fossile`. Les gemmes se répartissent en `PierrePrecieuse` (diamant, rubis, saphir, émeraude) et `PierreNonPrecieuse`, qui contient `PierreFine` (opale, améthyste). Les éléments chimiques descendent de `ElementChimique`.
- `rdfs:domain` et `rdfs:range` typent les propriétés. Par exemple, `aPourElement` va d'un minéral ou d'un minéraloïde vers un `ElementChimique`, et `estOriginaireDe` va d'une pierre vers un `Pays`. Les domaines composés sont exprimés avec `owl:unionOf`.
- `rdfs:label` et `rdfs:comment` documentent les classes et propriétés.

Classes OWL

- **Disjonction**: **owl:disjointWith** sépare `Mineral`, `Mineraloide`, `Roche` et `MatiereOrganique`, ainsi que `Rubis`/`Saphir` et `PierrePrecieuse`/`PierreNonPrecieuse`.

  owl:AllDisjointClasses déclare disjointes les espèces minérales (`Beryl`, `Calcite`, `Corindon`, `Diamant`, etc).
- **Restrictions** : `owl:someValuesFrom` décrit la composition de chaque espèce (un rubis a pour élément l'aluminium et l'oxygène). `owl:qualifiedCardinality` exprime qu'un minéral possède exactement un élément chimique.

- **Classe définie** : `MineralNatif` est `owl:equivalentClass` à l'intersection (`owl:intersectionOf`) d'un `Mineral` et d'une restriction de cardinalité 1 sur `aPourElement`. Le raisonneur classe ainsi automatiquement le diamant et le graphite (constitués de carbone seul).
- **Clé** : `owl:hasKey (codeISO)` sur `Pays` identifie chaque pays par son code ISO.
- **Classes alimentées par des règles** : `PierreTresDure`, `PierreDeCollection`, `RocheCarbonatee` et `PaysExportateur` sont vides à l'origine et peuplées par les règles SWRL (étape 4).

Propriétés d'objet

- `aPourConstituant` (roche → matière) est `owl:AsymmetricProperty` et `owl:IrreflexiveProperty` car une roche ne peut pas se constituer elle-même mais une roche peut être constitué d'autres roches.
- `aPourElement` (minéral → élément) est une `rdfs:subPropertyOf` de `contientElement`. (utile pour la chaine suivante)
- `contientElement` est définie par un `owl:propertyChainAxiom` : `aPourConstituant` ∘ `aPourElement`. Une roche contient donc les éléments de ses constituants.
- `estOriginaireDe` est une sous-propriété de `estTrouve` (un spécimen trouvé dans un pays).
- `estExportePar` et `estImportePar` ont pour inverses `exporte` et `importe` (`owl:inverseOf`). `exporteVers` (pays → pays) est déduite par la règle R6.
- `estPreserveDans` relie un fossile à la roche qui le contient.

Propriétés de données

`codeISO` (`xsd:string`), `dureteMohs` et `masseCarats` (`xsd:decimal`) sont toutes trois des `owl:FunctionalProperty` : chaque individu ne possède qu'une seule valeur. Elles servent de base aux règles SWRL sur la dureté et la masse (R2, R3).

Individus

Les 66 individus (pays, éléments chimiques, spécimens) sont déclarés `owl:NamedIndividual`. `owl:AllDifferent` garantit que les pays d'une part, et les éléments chimiques d'autre part, sont deux à deux distincts, ce qui compense l'absence de *Unique Name Assumption* en OWL. On retrouve, en général, au minimum deux individus par classe.

2.3 Démonstration du raisonnement OWL

L'ontologie obtenue après l'étape 2 (`RDF/RDFS/OWL`) étant consistante, des blocs de règles activables/désactivables via l'utilisation de commentaire ont été rajoutés dans cette étape pour tester des blocs insatisfiables et la logique de raisonnement `OWL`.

```
# Blocs insatisfiables pour l'etape 3 :

# ---- BLOC 1 : individu dans deux classes disjointes ----
#:PierreMysterieuse a :Rubis , :Saphir ;
#     rdfs:label "pierre mysterieuse"@fr ;
#     :estOriginaireDe :Myanmar .

# ---- BLOC 2 : classe insatisfiable (l'ontologie reste coherente) ----
# :RubisSynthetique a owl:Class ;
#      rdfs:subClassOf :Rubis , :Saphir ;
#      rdfs:label "rubis synthetique"@fr .

# ---- BLOC 3 : violation de propertyDisjointWith ----
# :Diamant_03 a :Diamant ;
#      rdfs:label "diamant n3"@fr ;
#      :estOriginaireDe :Botswana ;
#      :estImportePar :Botswana .

# ---- BLOC 4 : violation de FunctionalProperty ----
# :Bresil :codeISO "BRZ" .
```

Voici les différentes erreurs relevées par le raisonneur `HermiT` de `Protege` :

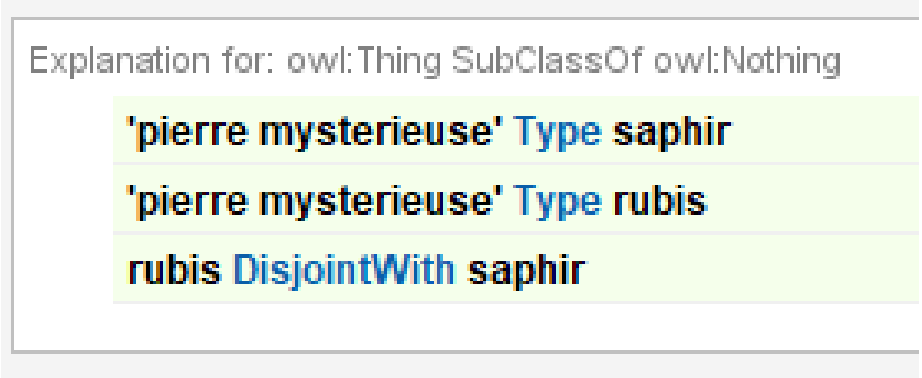


Figure 1: Incohérence du bloc 1 détectée par `HermiT` : individu dans deux classes disjointes

La propriété `disjointWith` dans la définition de la classe `Rubis` empêche toute instance de `Rubis` d'être aussi une instance de `Saphir` (et inversement). La pierre mysterieuse ne peut donc pas être les deux à la fois et doit être redéfinie ou supprimée.

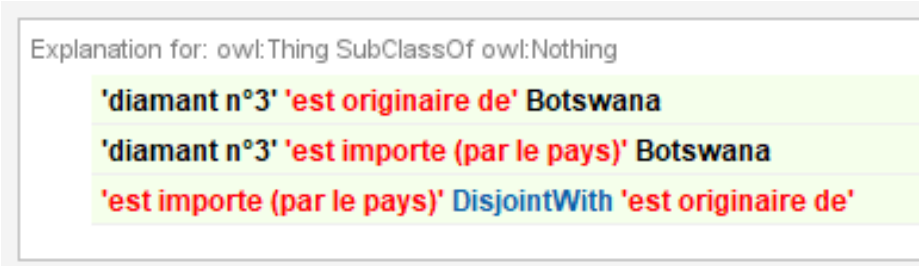


Figure 2: Incohérence du bloc 3 détectée par `HermiT` : propriétés disjointes

La propriété `estOriginaireDe` est définie par la propriété `propertyDisjointWith` avec `estImportePar`. Cette propriété indique qu'un individu originaire d'un pays ne peut pas être importé par ce même pays. Ici c'est le cas de `Diamant_03` qui est donc dans un état incohérent.

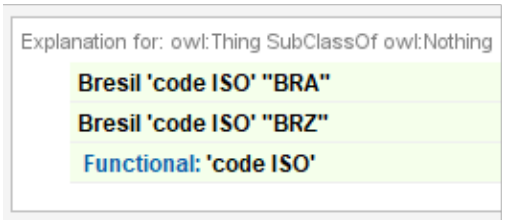


Figure 3: Incohérence du bloc 4 détectée par `HermiT` : `FunctionalProperty`

Cette incohérence est due à la propriété `FunctionalProperty` sur la propriété de données `codeISO`. Un pays ne peut être relié qu'à un seul `codeISO` faisant office d'ID. Ici, l'erreur vient de la tentative de relier `Bresil` à un deuxième `codeISO`.

`RubisSynthétique` dans le bloc 2 ne génère pas d'inconsistance car il n'est pas défini directement comme un `Rubis` ou un `Saphir` mais comme une classe héritant des deux. `HermiT` déduit par sa définition que la classe `RubisSynthetique` est équivalente à la classe `owl:Nothing`. Tant qu'aucun individu n'est créé pour cette classe, l'ontologie reste dans un état consistant.

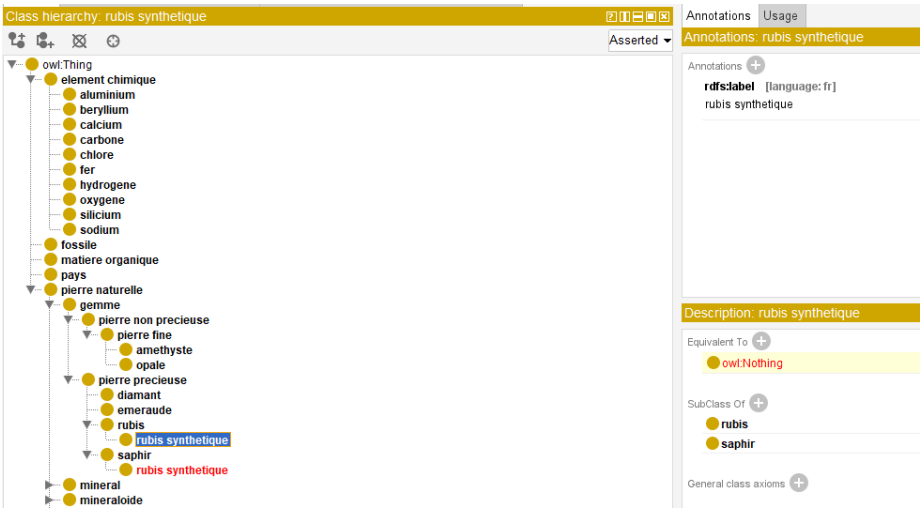


Figure 4: Pseudo-incohérence du bloc 2 : classe héritante de classes disjointes

Sur `Protege`, on peut voir cette équivalence déduite par le raisonneur.

2.4 Décidabilité `SWRL`

Les règles `SWRL` (R1 à R8) sont exécutées par le moteur `Drools` de Protégé sur les individus nommés de l'ontologie. Elles restent simples (conjonctions

d'atomes, comparaisons numériques), ce qui limite le risque d'indécidabilité du `SWRL` général : les inférences se terminent et peuvent être vérifiées par des requêtes.

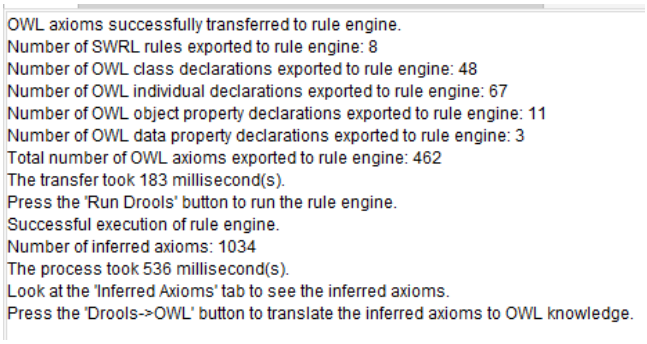


Figure 5: Résultat des inférences créées après `Run Drools`

Dans le terminal de l'onglet `SWRL`, il était affiché que 1034 axiomes avaient été générés après l'application des logiques `OWL/SWRL`. En réalité, de nombreuses erreurs liées à l'interprétation de `propertyDisjointWith` sur `estOriginaireDe` et `estImportePar` sont apparues. `estOriginaireDe` était devenue une sous-propriété de `estImportePar`, générant sur chaque individu dont l'origine était définie une incohérence vis-à-vis de leur statut disjoint. Ainsi, environ 900 axiomes ont réellement été générés après le débug. Pour vérifier l'efficacité des règles `SWRL`, voici quelques `DL Queries` effectuées sur des instances ajoutées explicitement pour vérifier leur fonctionnement :

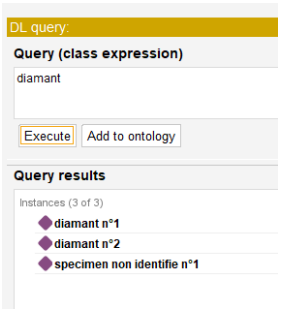


Figure 6: Query démontrant R1

R1 (identification d'un diamant).
Tout minéral dont l'un des éléments est le carbone et dont la dureté de Mohs est supérieure ou égale à 10 est classé `Diamant`. La requête `diamant` renvoie les deux diamants déclarés, mais aussi le specimen non identifie n°1, qui n'était pas typé comme diamant. Le specimen non identifie n°2, bien que similaire dans sa composition n'a une dureté de Mohs de 1.5 et n'est donc pas déclaré `Diamant`.

```
R1 : mineral(?m) ^ 'a pour element'(?m, ?e) ^ carbone
   (?e) ^ 'durete (echelle de Mohs)'(?m, ?d) ^ swrlb:
   greaterThanOrEqual(?d, "10"^^xsd:decimal) ->
   diamant(?m)
```

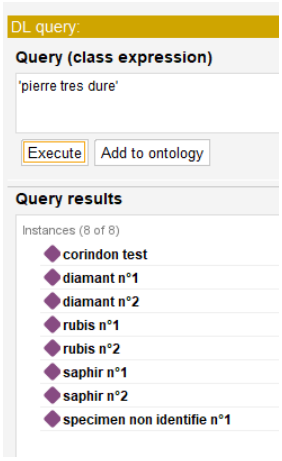


Figure 7: Query démontrant R2

R2 (pierre très dure). Une gemme de dureté de Mohs supérieure ou égale à 9 est classée `PierreTresDure`. La requête retourne 8 instances : diamants, rubis, saphirs et corindon de test. Le specimen non identifié n°1 en fait partie grâce à R1, ce qui montre l'enchaînement des règles.

```
R2 : gemme(?g) ^ 'durete (echelle de Mohs)'(?g, ?d) ^
   swrlb:greaterThanOrEqual(?d, "9"^^xsd:decimal) -> '
   pierre tres dure'(?g)
```

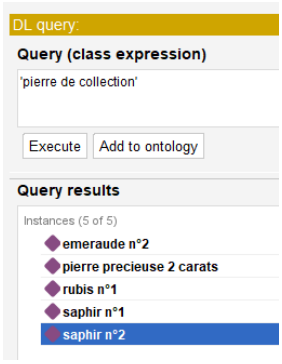


Figure 8: Query démontrant R3

R3 (pierre de collection). Une pierre précieuse de 2 carats ou plus est classée `PierreDeCollection`. Cinq individus sont retournés, dont la pierre precieuse 2 carats, ajoutée pour tester le seuil. Les pierres de moins de 2 carats ne sont pas concernées.

```
R3 : 'pierre precieuse'(?p) ^ 'masse (carats)'(?p, ?m)
     ^ swrlb:greaterThanOrEqual(?m, "2.0"^^xsd:decimal)
     -> 'pierre de collection'(?p)
```

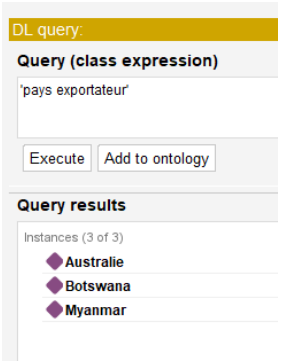


Figure 9: Query démontrant R7

R7 (pays exportateur). Un pays qui exporte vers un autre pays est classé `PaysExportateur`. Cette règle s'appuie sur R6, qui déduit la propriété `exporteVers` à partir des propriétés `estExportePar` et `estImportePar` d'une même pierre. La requête identifie l'Australie, le Botswana et le Myanmar.

```
R7 : pays(?x) ^ 'exporte vers'(?x, ?y) -> 'pays
   exportateur'(?x)
```

2.5 Validation SHACL

Les shapes `SHACL` complètent le raisonnement `OWL`. `OWL` fonctionne en monde ouvert : une valeur absente n'est pas une erreur, et une dureté de 11 est acceptée puisque `xsd:decimal` n'impose pas de borne. `SHACL` raisonne en monde fermé et vérifie que les données sont complètes et bien formées (cardinalités, types, formats, bornes). Il distingue aussi deux niveaux de gravité : `Violation` pour les contraintes strictes et `Warning` pour les recommandations.

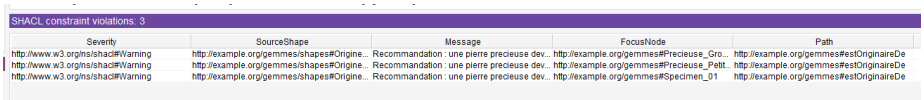


Figure 10: Violations détectées par les shapes `SHACL` AVANT l'ajout de violations tests

Sur les données de l'ontologie, aucune contrainte stricte n'est violée : seuls 3 avertissements apparaissent. Ils proviennent de la recommandation selon laquelle une pierre précieuse devrait avoir un pays d'origine documenté (propriété `estOriginaireDe`). Ces avertissements sont volontaires et non bloquants. Ils signalent seulement une information manquante.

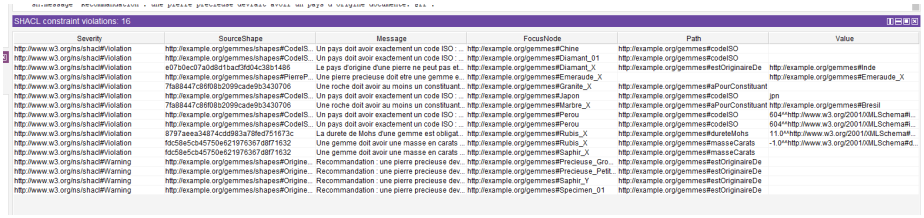


Figure 11: Violations détectées par les shapes `SHACL` APRÈS l'ajout de violations tests

Après l'ajout d'individus volontairement erronés, le validateur remonte 16 résultats : 12 `Violation` et 4 `Warning` (le nouvel avertissement concerne `Saphir_Y`). Chaque violation test est détectée sur le bon nœud et la bonne propriété :

Nœud	**Problème détecté**
`Chine`	code ISO absent
`Japon`	code ISO au format incorrect (`jpn`)
`Perou`	code ISO saisi comme entier (`604`) au lieu d'une chaîne, ce qui produit deux résultats
`Diamant_X`	pays d'origine incompatible avec un autre pays lié à la pierre (`Inde`)
`Emeraude_X`	pierre précieuse ne respectant pas la contrainte de sa shape dédiée
`Granite_X`, `Marbre_X`	roche sans aucun constituant (`aPourConstituant`)
`Rubis_X`	dureté de 11 (hors de l'échelle de Mohs, de 1 à 10) et masse de -1 carat
`Saphir_X`	masse en carats absente

Ces cas montrent la complémentarité des deux approches : aucun d'eux ne rendrait l'ontologie incohérente pour `HermiT`, alors que `SHACL` les signale tous. À l'inverse, la contrainte sur `estOriginaireDe` et `estImportePar`, détectée en `OWL` à l'étape 3 par `propertyDisjointWith`, est ici reformulée en contrainte de validation, ce qui permet de la signaler sans bloquer le raisonneur.

2.6 Bonnes pratiques en Web Sémantique

ll'objectif de ce section est d'explorer les bonnes pratiques mises en place pour le développement de l'ontologie de manière à ce qu'on puisse construire une

ontologie bien structurée et utilisable dans d'autres exemples.

Tout d'abord, entre les bonnes pratiques suggérées existent : La réutilisation de vocabulaires existants comme foaf, schema.org, dbo, etc... ; la modularité et la séparation des questions et inquiétudes ; l'utilisation correcte de la syntaxe : is-a, instance-of, part-of, sub-class-of, etc. et les conventions de nommage.

Afin de valider l'ontologie, on a développé quelques questions avec l'objectif de valider l'ontologie et d'observer si elle est bien construite et sa capacité à donner des résultats selon les requêtes demandées.

Questions de compétence en langage naturel
à continuation, il est possible d'observer des questions écrites en langage naturel traduites au langage SPARKQL pour leur validation.

- Quelles pierres précieuses proviennent du pays Myanmar et quelle est leur masse en carats?

```
PREFIX : <http://example.org/gemmes#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema
   #>
SELECT ?nomPierre ?masse
WHERE {
  ?pierre a :PierrePrecieuse ;
          :estOriginaireDe :Myanmar ;
          :masseCarats ?masse ;
          rdfs:label ?nomPierre .
}
```

NomPierre↑	Masse↑
saphir n°1	3.0
rubis n°2	1.8
rubis n°1	2.5

- Quels minéraux ont une dureté supérieure ou égale à 8 sur l'échelle de Mohs et quelle est leur composition chimique?

```
PREFIX : <http://example.org/gemmes#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema
   #>
SELECT ?nomMineral ?durete ?element
WHERE {
  ?mineral a :Mineral ;
           :dureteMohs ?durete ;
           :aPourElement ?element ;
```

```
            rdfs:label ?nomMineral .
  FILTER (?durete >= 8.0)
}
```

NomMineral↑	Durete↑	Element↑
diamant n°2	10.0	<http://dbpedia.org/resource/Carbon>
diamant n°1	10.0	<http://dbpedia.org/resource/Carbon>
saphir n°2	9.0	<http://dbpedia.org/resource/Oxygen>
saphir n°2	9.0	<http://dbpedia.org/resource/Aluminium>
saphir n°1	9.0	<http://dbpedia.org/resource/Oxygen>
saphir n°1	9.0	<http://dbpedia.org/resource/Aluminium>
rubis n°2	9.0	<http://dbpedia.org/resource/Oxygen>
rubis n°2	9.0	<http://dbpedia.org/resource/Aluminium>
rubis n°1	9.0	<http://dbpedia.org/resource/Oxygen>
rubis n°1	9.0	<http://dbpedia.org/resource/Aluminium>

- Quelles roches ont pour constituant un minéraux contenant du silicium (silicon)?

```
PREFIX : <http://example.org/gemmes#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema
   #>
PREFIX dbr: <http://dbpedia.org/resource/>
SELECT ?nomRoche ?nomConstituant
WHERE {
  ?roche a :Roche ;
         :aPourConstituant ?constituant ;
         rdfs:label ?nomRoche .
  ?constituant :aPourElement dbr:Silicon ;
               rdfs:label ?nomConstituant .
}
```

NomRoche↑	NomConstituant↑
obsidienne n°1	verre volcanique n°1
geode d'amethyste n°1	amethyste n°1
geode d'amethyste n°2	amethyste n°2
obsidienne n°2	verre volcanique n°2

- Quelles pierres naturelles sont exportées par Brésil et l'Australie?

```
PREFIX : <http://example.org/gemmes#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema
   #>

SELECT ?nomPierre ?nomPays
WHERE {
  ?pierre a :PierreNaturelle ;
          :estExportePar ?pays ;
          rdfs:label ?nomPierre .
  ?pays rdfs:label ?nomPays .
  FILTER (?pays IN (:Bresil, :Australie))
}
```

NomPierre↑	NomPays↑
diamant n°2	Australie

L'ontologie développée, on a utilisé correctes utilisations taxonomiques selon la relation des axiomes pour éviter des inconsistances. Un exemple à évoquer, c'est l'utilisation de la relation `:aPourConstituant` qui permet de relier la composition des roches avec des minéraux, la minéralogie et la matière organique, à la place d'spécifié que ces matériaux soient une sous classe de `:Roche`.

```
:aPourConstituant a owl:ObjectProperty, owl:
   AsymmetricProperty, owl:IrreflexiveProperty ;
    rdfs:domain :Roche ;
    rdfs:range [ a owl:Class ;
                 owl:unionOf ( :Mineral :Mineraloide :
                    MatiereOrganique ) ] ;
    rdfs:label "a pour constituant"@fr ;
    rdfs:comment "Relie une roche aux matieres qui la
       constituent."@fr .
```

En plus, on a appliqué la réutilisation des ontologies existantes avec l'utilisation de l'ontologie DBpedia dans la section des éléments chimiques, ce qui nous a évité de créer , une nouvelle ontologie dès zéro avec des éléments chimiques.

```
:ElementChimique a owl:Class ;
    owl:equivalentClass dbo:ChemicalElement;
    rdfs:label "element chimique"@fr ;
    rdfs:comment "Element du tableau periodique
       entrant dans la composition d'un mineral."@fr .
```

3 Conclusion

Ce projet détaille la conception, la construction et la validation d'une base de connaissances sémantiques axée sur le domaine de la gemmologie et de la minéralogie.

Le travail démontre l'application de méthodologies formelles pour représenter la composition chimique, les propriétés physiques et l'origine géographique de diverses pierres naturelles.

On a développé une architecture ontologique (TBOX et ABOX) avec des hiérarchies de classes précises (minéraux, roches, pierres précieuses), assurant la disjonction entre des concepts incompatibles et définissant des restrictions de domaine et de rang.

Les propriétés fonctionnelles et les axiomes des chaînes de propriété ont été incorporés pour en déduire des relations indirectes.

Par la suite, un raisonnement OWL de logique descriptive (DL) a été appliqué pour valider la cohérence globale du modèle et exécuter le classement automatique des instances.
Pour étendre la capacité de déduction au-delà d'OWL, des règles SWRL orientées vers le classement dynamique des individus et la propagation des propriétés ont été mises en place.

Parallèlement, des contraintes structurelles ont été mises en place via SHACL pour auditer la base de connaissances, garantissant que les données respectent les schémas logiques requis avant leur validation finale.

Dans la phase finale, le modèle a été affiné en suivant les standards du Web sémantique. Le principe de modularité et de réutilisation des vocabulaires a été appliqué en intégrant les ressources standardisées de DBpedia pour les éléments chimiques.

Enfin, l'efficacité de l'ontologie a été évaluée en structurant des questions de compétence (Competency Questions) exécutées à l'aide de requêtes SPARQL, et en intégrant des modèles de conception ontologique (ODPs) de la littérature pour résoudre des problèmes complexes de modélisation.